NLP Notes

Module 1: Introduction to NLP

---

1. What is Natural Language Processing (NLP)?

Natural Language Processing (NLP) is a branch of **Artificial Intelligence (AI)** that enables computers to understand, interpret, process, and generate human language.

Simply:

NLP = Teaching computers to understand and communicate in human languages like English, Hindi, Marathi, etc.

Examples

- Google Translate
- ChatGPT
- Siri & Alexa
- Grammarly
- Gmail Smart Reply

---

2. Why NLP?

Computers understand **numbers**, but humans communicate using **language**.

NLP acts as a **bridge between humans and computers**.

Benefits

- Understand human language
- Automate text processing
- Improve customer support
- Analyze large amounts of text
- Enable voice assistants and chatbots

---

3. History of NLP

Year	Development

Year	Development
1950	Alan Turing proposed the Turing Test
1954	First Machine Translation experiment
1960s	Rule-Based NLP systems
1980s	Statistical NLP introduced
2010s	Deep Learning revolution
2018+	Transformer models (BERT, GPT)
Today	Large Language Models (LLMs) like ChatGPT

---

4. Evolution of NLP

1. Rule-Based NLP

- Uses manually written grammar rules.
- No learning from data.

Example

IF sentence contains "good"
→ Positive

---

2. Statistical NLP

- Learns patterns from data using probability.
- More flexible than rule-based systems.

Example

- Naive Bayes
- Hidden Markov Models

---

3. Deep Learning NLP

- Uses neural networks.
- Learns automatically from large datasets.
- State-of-the-art approach.

Examples

- BERT

- GPT
- Llama

---

5. NLP Applications

Common real-world applications:

- Machine Translation
- Chatbots
- Virtual Assistants
- Sentiment Analysis
- Spam Detection
- Text Summarization
- Question Answering
- Speech Recognition
- Text Classification
- Grammar Checking
- Autocomplete
- Search Engines
- Information Extraction

---

6. NLP Pipeline

The NLP pipeline is the sequence of steps used to process text.

Input Text
↓
Text Cleaning
↓
Tokenization
↓
Stopword Removal
↓
Stemming/Lemmatization
↓
Feature Extraction
↓
Model Training
↓
Prediction

Example

Input:

"I love learning NLP!"

After processing:

["love", "learn", "nlp"]

---

7. Levels of Language Processing

1. Lexical Level

Studies individual words.

Example:

Apple

---

2. Syntactic Level

Checks grammar and sentence structure.

Example:

She is playing football.

---

3. Semantic Level

Understands the meaning of the sentence.

Example:

I deposited money in the bank.

"Bank" means a financial institution.

---

4. Pragmatic Level

Understands meaning using context.

Example:

Can you open the window?

Meaning:

It is a request, not a question about ability.

---

5. Discourse Level

Understands relationships between multiple sentences.

Example:

Rahul bought a car.
He loves it.

"It" refers to the **car**.

---

8. NLP Challenges

NLP is difficult because human language is complex.

Common challenges:

- Ambiguity (same word, multiple meanings)
- Different languages
- Slang and abbreviations
- Spelling mistakes
- Grammar variations
- Sarcasm
- Context understanding
- Named entities
- Multiple meanings of the same sentence

Example

I saw a bat.

"Bat" could mean:

- Animal
- Cricket bat

Module 2: Linguistics for NLP

---

1. Natural Language

A **natural language** is a language spoken and written by humans.

Examples

- English
- Hindi
- Marathi
- French

Unlike programming languages, natural languages have grammar, ambiguity, and context.

---

2. Corpus

A **Corpus** is a large collection of text or speech data used to train and evaluate NLP models.

Examples

- Wikipedia articles
- News articles
- Books
- Tweets

Corpus = Collection of text data

---

3. Vocabulary

Vocabulary is the set of all unique words in a dataset.

Example

Sentence:

I love NLP. I love AI.

Vocabulary:

I, love, NLP, AI

Vocabulary Size = **4**

---

4. Token

A **Token** is the smallest unit obtained after splitting text.

Example

Sentence:

I love NLP

Tokens:

["I", "love", "NLP"]

---

5. Lexicon

A **Lexicon** is a dictionary of words along with their meanings and linguistic information.

It may contain:

- Meaning
- Part of Speech (POS)
- Pronunciation
- Synonyms

---

6. Morphology

Morphology is the study of the structure and formation of words.

It explains how words are built from smaller parts.

Example

Unhappiness

Split into:

Un + Happy + Ness

---

7. Morpheme

A **Morpheme** is the smallest meaningful unit of a word.

Example

Unhappy

Morphemes:

- Un
- Happy

---

8. Stem

A **Stem** is the base form obtained after removing prefixes or suffixes.

Example

Playing → Play

---

9. Root Word

A **Root Word** is the original core word from which other words are formed.

Example

Act

Derived words:

- Action
- Actor
- Active

---

10. Lemma

A **Lemma** is the dictionary (base) form of a word.

Examples

Running → Run
Better → Good
Children → Child

Lemmatization produces meaningful dictionary words.

---

11. Syntax

Syntax is the study of sentence structure and grammar.

It checks whether a sentence is grammatically correct.

Correct

She is reading a book.

Incorrect

Reading she book is.

---

12. Grammar

Grammar is the set of rules for forming correct sentences.

It defines:

- Word order
- Tenses
- Agreement
- Sentence structure

---

13. Phrase Structure

A **Phrase Structure** shows how words combine to form phrases.

Example

The smart student

Noun Phrase:

The + smart + student

---

14. Constituency

Constituency groups words that function together as one unit.

Example

The boy plays football.

Groups:

- The boy
- plays football

---

15. Dependency Grammar

Dependency Grammar shows how words depend on each other.

Each word is connected to another related word.

Example

She eats apples.

- eats → Main verb
- She → depends on eats
- apples → depends on eats

---

16. Parse Tree

A **Parse Tree** is a tree showing the grammatical structure of a sentence.

Example

Sentence
├── Noun Phrase
└── Verb Phrase

Used to analyze sentence structure.

---

17. Dependency Tree

A **Dependency Tree** represents dependency relationships between words.

Example

eats
├── She
└── apples

Focuses on word relationships instead of phrases.

---

18. Semantics

Semantics is the study of the meaning of words and sentences.

Example

The cat is sleeping.

Meaning:
A cat is currently asleep.

---

19. Meaning Representation

Meaning Representation is a structured way of representing sentence meaning so computers can understand it.

It converts text into logical or semantic forms.

---

20. Lexical Semantics

Lexical Semantics studies the meanings and relationships between words.

It includes:

- Synonyms
- Antonyms
- Hyponyms
- Hypernyms
- Polysemy

---

21. Word Sense

A **Word Sense** is a specific meaning of a word depending on context.

Example

Bank

Meaning 1:
Financial institution

Meaning 2:
River bank

---

22. Polysemy

Polysemy means one word has multiple related meanings.

Example

Head

- Head of a person
- Head of a company

---

23. Synonym

Synonyms are words with similar meanings.

Examples

- Happy → Joyful
- Big → Large
- Fast → Quick

---

24. Antonym

Antonyms are words with opposite meanings.

Examples

- Hot ↔ Cold
- Up ↔ Down
- Rich ↔ Poor

---

25. Hyponym

A **Hyponym** is a specific type of a broader category.

Examples

- Dog → Animal
- Rose → Flower
- Apple → Fruit

Dog is a hyponym of Animal.

---

26. Hypernym

A **Hypernym** is the broader category that includes specific words.

Examples

- Animal
- Fruit
- Vehicle

Animal is the hypernym of Dog.

---

27. Pragmatics

Pragmatics studies the intended meaning of a sentence based on context.

Example

It's cold here.

Actual meaning:

Please close the window.

---

28. Context

Context is the surrounding information that helps determine meaning.

Example

He sat on the bank.

- Near a river → River bank
- Near money → Financial bank

---

29. Discourse

Discourse is the study of multiple connected sentences.

It helps understand the overall meaning of a conversation or paragraph.

Example

Rahul bought a laptop.
He uses it every day.

"It" refers to the laptop.

---

30. Coreference

Coreference occurs when two or more words refer to the same entity.

Example

Priya lost her phone.
She found it later.

- She → Priya
- It → Phone

---

31. Anaphora

Anaphora is a type of coreference where a word refers to something mentioned earlier.

Example

Ravi bought a bike.
He rides it daily.

- He → Ravi
- It → Bike

---

32. Ellipsis

Ellipsis is the omission of words that are understood from the context.

Example

I like tea, and she does too.

Full meaning:

I like tea, and she likes tea too.

Module 3: Text Preprocessing

What is Text Preprocessing?

Text Preprocessing is the process of cleaning and preparing raw text before giving it to an NLP model.

Purpose

- Remove unwanted data
- Make text consistent
- Improve model accuracy

Example

Raw Text:

"Hello!!! Visit https://abc.com □"

After Preprocessing:

hello visit

---

1. Text Cleaning

Text Cleaning is the process of removing unwanted information from text.

It includes:

- Lowercasing
- Removing HTML
- Removing URLs
- Removing Emojis
- Removing Numbers
- Removing Punctuation
- Removing Special Characters
- Removing Extra Spaces

---

2. Lowercasing

Convert all letters into **lowercase**.

Example

Before:

I Love NLP

After:

i love nlp

Benefit

- Treats "Apple" and "apple" as the same word.

---

3. Removing HTML Tags

Remove HTML elements from text.

Example

Before:

Hello World

After:

Hello World

---

4. Removing URLs

Remove website links.

Example

Before:

Visit https://google.com

After:

Visit

---

5. Removing Emojis

Remove emoji characters.

Example

Before:

I love NLP □❤□

After:

I love NLP

---

6. Removing Numbers

Remove numeric values if they are not useful.

Example

Before:

I have 2 books

After:

I have books

---

7. Removing Punctuation

Remove punctuation marks.

Example

Before:

Hello, World!

After:

Hello World

Punctuation:

. , ! ? : ; ' "

---

8. Removing Special Characters

Remove unnecessary symbols.

Example

Before:

Python@#NLP$

After:

---

9. Removing Extra Spaces

Remove multiple spaces.

Example

Before:

I love NLP

After:

I love NLP

---

10. Tokenization

Tokenization is the process of splitting text into smaller units called **tokens**.

Example

Sentence:

I love NLP.

Tokens:

["I", "love", "NLP"]

---

11. Sentence Tokenization

Split a paragraph into sentences.

Example

Before:

I love NLP. It is interesting.

After:

Sentence 1: I love NLP.

Sentence 2: It is interesting.

---

12. Word Tokenization

Split a sentence into words.

Example

Sentence:

I love AI

Tokens:

["I", "love", "AI"]

---

13. Character Tokenization

Split text into individual characters.

Example

Word:

NLP

Tokens:

["N", "L", "P"]

---

14. Normalization

Normalization converts text into a standard and consistent format.

Example

Before:

Running, RUNNING, running

After:

running

---

15. Case Folding

Convert all uppercase letters into lowercase.

Example

Before:

APPLE

After:

apple

Case folding is a type of text normalization.

---

16. Unicode Normalization

Convert different Unicode representations of the same character into a standard form.

Example

Different forms of:

é

After normalization:

é

Ensures text is stored consistently.

---

17. Text Normalization

Convert text into a clean, standardized format.

It may include:

- Lowercasing
- Removing punctuation

- Expanding contractions
- Fixing repeated characters

Example

Before:

I'm soooo Happy!!!

After:

I am so happy

---

18. Stop Words

Stop Words are very common words that usually do not add much meaning.

Examples

- is
- am
- the
- a
- an
- in
- on
- of
- to

---

19. Stop Word Removal

Remove stop words from text.

Example

Before:

I am learning NLP

After:

learning NLP

Benefit

- Reduces unnecessary words.
- Makes processing faster.

---

20. Stemming

Stemming reduces words to their root-like form by removing prefixes or suffixes.

The resulting word **may not be a valid dictionary word**.

Examples

Playing → Play
Running → Run
Studies → Studi

---

21. Porter Stemmer

The **Porter Stemmer** is the most popular stemming algorithm.

It removes common suffixes.

Example

Connected → Connect
Playing → Play

---

22. Snowball Stemmer

An improved version of the Porter Stemmer.

Features

- Faster
- Supports multiple languages
- More accurate

Example

Running → Run

---

23. Lancaster Stemmer

A more aggressive stemming algorithm.

It removes more characters than Porter.

Example

Maximum → Maxim

Faster but can over-stem words.

---

24. Lemmatization

Lemmatization converts words into their dictionary (lemma) form using vocabulary and grammar.

The output is always a meaningful word.

Examples

Running → Run
Better → Good
Children → Child

---

25. POS-based Lemmatization

POS-based Lemmatization uses the **Part of Speech (POS)** of a word to find the correct lemma.

Example

Word:

Saw

As a **Verb**:

Saw → See

As a **Noun**:

Saw → Saw

Using POS gives more accurate lemmatization.

Module 4: Text Representation

What is Text Representation?

Text Representation is the process of converting text into **numbers (vectors)** so that machine learning and deep learning models can understand it.

Text → Numbers → Model

---

1. Bag of Words (BoW)

Bag of Words (BoW) represents a document by counting how many times each word appears.

It ignores:

- Word order
- Grammar
- Context

Example

Sentence:

I love NLP
I love AI

Vocabulary:

[I, love, NLP, AI]

Vectors:

Sentence	I	love	NLP	AI
I love NLP	1	1	1	0
I love AI	1	1	0	1

---

2. One-Hot Encoding

Each word is represented by a binary vector.

Only one position is **1**, others are **0**.

Vocabulary:

[Cat, Dog, Bird]

Word	Vector
Cat	[1,0,0]
Dog	[0,1,0]
Bird	[0,0,1]

Limitation

- Very sparse vectors
- No meaning between words

---

3. Count Vectorizer

A **Count Vectorizer** converts text into a matrix of word counts.

Example

Sentences:

I love AI
AI is amazing

Vocabulary:

[I, love, AI, is, amazing]

Count Matrix:

Sentence	I	love	AI	is	amazing
I love AI	1	1	1	0	0
AI is amazing	0	0	1	1	1

---

4. Document-Term Matrix (DTM)

A **Document-Term Matrix** stores the frequency of each word in every document.

- Rows → Documents
- Columns → Words
- Values → Word counts

DTM is commonly created using Count Vectorizer.

---

5. Term Frequency (TF)

TF measures how often a word appears in a document.

Formula

TF = (Word Count) / (Total Words)

Example

Document:

AI AI NLP

TF(AI)

2 / 3 = 0.67

---

6. Inverse Document Frequency (IDF)

IDF measures how important a word is across all documents.

Common words receive **lower scores**.

Rare words receive **higher scores**.

Formula

IDF = log(Total Documents / Documents containing the word)

---

7. TF-IDF

TF-IDF combines TF and IDF to measure the importance of a word.

Formula

TF-IDF = TF × IDF

Benefit

- Reduces the importance of common words.
- Highlights important words.

---

8. N-grams

An **N-gram** is a sequence of **N consecutive words**.

Used to capture word order.

---

9. Unigram

A **Unigram** contains **1 word**.

Sentence:

I love NLP

Output:

[I]
[love]
[NLP]

---

10. Bigram

A **Bigram** contains **2 consecutive words**.

Sentence:

I love NLP

Output:

[I love]
[love NLP]

---

11. Trigram

A **Trigram** contains **3 consecutive words**.

Sentence:

I love learning

Output:

[I love learning]

---

12. N-gram Models

An **N-gram Model** predicts the next word using the previous **N−1 words**.

Example

I love → NLP

Used in:

- Autocomplete
- Next-word prediction
- Speech recognition

---

13. Skip-grams

A **Skip-gram** allows skipping words while creating word pairs.

Sentence:

I love learning NLP

Possible pairs:

(I, love)
(I, learning)
(love, NLP)

Used in **Word2Vec Skip-Gram**.

Module 5: Word Embeddings

What are Word Embeddings?

Word Embeddings are dense numerical vectors that represent the meaning of words.

Words with similar meanings have similar vectors.

Word → Dense Vector

---

1. Distributed Representation

A **Distributed Representation** represents a word using many numerical values instead of a single number.

Example:

King → [0.24, -0.51, 0.88, ...]

Each value captures different semantic features.

---

2. Dense Embeddings

A **Dense Embedding** is a vector where most values are non-zero.

Example

Cat

Dense Vector:

[0.21, -0.47, 0.88, 0.12]

Unlike One-Hot Encoding, dense embeddings are compact and meaningful.

---

3. Word Embeddings

Word embeddings convert words into vectors that capture semantic meaning.

Example

- King ≈ Queen
- Car ≈ Vehicle
- Dog ≈ Puppy

Similar words have vectors close to each other.

---

4. Distributional Hypothesis

The **Distributional Hypothesis** states:

"Words that appear in similar contexts tend to have similar meanings."

Example

The cat drinks milk.
The dog drinks milk.

"Cat" and "Dog" have similar contexts, so their embeddings become similar.

---

5. Word2Vec

Word2Vec is a popular algorithm that learns word embeddings from text.

It has two models:

- CBOW
- Skip-Gram

---

6. CBOW (Continuous Bag of Words)

CBOW predicts the **target word** using surrounding context words.

Example

I ___ NLP

Context:

I, NLP

Prediction:

love

---

7. Skip-Gram

Skip-Gram predicts the **surrounding words** using the target word.

Example

Target:

love

Predict:

I
NLP

---

8. Negative Sampling

Negative Sampling speeds up Word2Vec training by updating only a few selected words instead of the entire vocabulary.

Benefit

- Faster training
- Less computation

---

9. Hierarchical Softmax

Hierarchical Softmax speeds up prediction by organizing words in a binary tree.

Instead of checking every word, it follows a path through the tree.

Benefit

- Efficient for large vocabularies.

---

10. GloVe

GloVe (Global Vectors) learns word embeddings using **global word co-occurrence statistics**.

It captures overall relationships between words.

Example

- Paris ↔ France
- Tokyo ↔ Japan

---

11. FastText

FastText represents words using **character-level subwords**.

It works well for:

- Rare words
- Misspelled words
- Morphologically rich languages

Example

playing

Subwords:

play
lay
ing

---

12. Static vs Contextual Embeddings (Conceptual)

Static Embeddings

A word always has **one fixed vector**, regardless of context.

Example

Bank → Same vector everywhere.

Examples:

- Word2Vec
- GloVe
- FastText

Contextual Embeddings

The vector changes depending on the sentence.

Example

River bank

and

Bank account

Different meanings → Different vectors.

Examples:

- BERT
- GPT

---

13. Embedding Similarity

Embedding Similarity measures how close two word vectors are.

More similar vectors → More similar meanings.

---

14. Cosine Similarity

Cosine Similarity measures the angle between two vectors.

Range

- **1** → Exactly similar
- **0** → Unrelated
- **−1** → Opposite direction

Used most often for comparing embeddings.

---

15. Euclidean Distance

Euclidean Distance measures the straight-line distance between two vectors.

- Smaller distance → More similar
- Larger distance → Less similar

---

16. Dot Product

The **Dot Product** measures how much two vectors point in the same direction.

- Larger value → More similar
- Smaller value → Less similar

Often used inside neural networks and attention mechanisms.

Module 6: Language Modeling

What is a Language Model?

A **Language Model (LM)** predicts the **next word** or calculates the probability of a sequence of words.

Goal: Learn how words are likely to appear together.

Example

I love → NLP

The model predicts **"NLP"** as the next word.

---

1. Language Model

A **Language Model** estimates how likely a sentence is.

Example

I love machine learning.

High probability □

Learning machine love I.

Low probability □

---

2. Probability of Sentence

A language model assigns a probability to an entire sentence.

Example

P(I love NLP)

Higher probability = More natural sentence.

---

3. Chain Rule

The **Chain Rule** breaks a sentence probability into probabilities of individual words.

Formula

$P(w1,w2,w3)=P(w1)\times P(w2|w1)\times P(w3|w1,w2)$

Example

I love NLP
P(I love NLP)
= P(I) × P(love|I) × P(NLP|I,love)

---

4. Markov Assumption

The **Markov Assumption** assumes the next word depends only on a few previous words, not the entire sentence.

Example

Instead of:

P(NLP | I really love studying)

Use:

P(NLP | love studying)

This makes computation easier.

---

5. Statistical Language Models

These models predict words using **probability and word frequencies**.

Examples:

- Unigram
- Bigram
- Trigram
- N-gram

---

6. Unigram Model

A **Unigram Model** assumes every word is independent.

Example

I love NLP
P(I) × P(love) × P(NLP)

No context is used.

---

7. Bigram Model

A **Bigram Model** uses **one previous word**.

Example

I love NLP
P(I) × P(love|I) × P(NLP|love)

---

8. Trigram Model

A **Trigram Model** uses **two previous words**.

Example

I love NLP
P(I) × P(love|I) × P(NLP|I,love)

---

9. N-gram Language Model

An **N-gram Language Model** predicts a word using the previous **N−1 words**.

Examples:

- Unigram → 1 word
- Bigram → 2 words
- Trigram → 3 words

---

10. Smoothing

Smoothing assigns a small probability to unseen word combinations.

Without smoothing:

Unknown sentence

Probability = **0**

With smoothing:

Probability > **0**

---

11. Laplace Smoothing

Also called **Add-One Smoothing**.

Adds **1** to every word count.

Benefit

- Prevents zero probabilities.
- Very simple to use.

---

12. Add-k Smoothing

Instead of adding **1**, add a small value **k** (such as 0.5).

Benefit

- More flexible than Laplace Smoothing.
- Reduces overestimation.

---

13. Good-Turing Smoothing

Good-Turing estimates probabilities for **unseen words** using the frequencies of rare words.

Idea

Rare events help estimate unseen events.

---

14. Backoff

If a higher-order N-gram is missing, **Backoff** uses a lower-order model.

Example

Trigram not found:

I love NLP

↓

Use Bigram

↓

If needed, use Unigram.

---

15. Interpolation

Interpolation combines multiple N-gram models.

Example

Final prediction uses:

- Unigram
- Bigram
- Trigram

Together for better accuracy.

---

16. Perplexity

Perplexity measures how well a language model predicts text.

- Lower Perplexity → Better model
- Higher Perplexity → Poor model

It is one of the most common evaluation metrics for language models.

Module 7: Sequence Modeling

What is Sequence Modeling?

Sequence Modeling is the process of learning patterns from **ordered data**, where the order of elements matters.

Examples

- Sentences
- Speech
- Time series
- DNA sequences

---

1. Sequential Data

Sequential Data is data where the order of items is important.

Example

I love NLP

Changing the order changes the meaning.

---

2. Sequence

A **Sequence** is an ordered list of elements.

Example

["I", "love", "AI"]

---

3. Context Window

A **Context Window** is the set of nearby words used to understand or predict a word.

Example

Sentence:

I love learning NLP

For **learning**, the context may be:

love, NLP

---

4. Time Dependency

Time Dependency means the current output depends on previous inputs.

Example

I am eating ...

The next word depends on the earlier words.

---

5. Sequence Models

Sequence Models process data one element at a time while remembering previous information.

Used for:

- Machine Translation
- Text Generation
- Speech Recognition
- Sentiment Analysis

---

6. Feedforward NLP Models

Feedforward Models process each input independently.

They **do not remember previous words**.

Limitation

Cannot capture long-term context.

---

7. Recurrent Neural Network (RNN)

An **RNN** processes sequences by passing information from one time step to the next.

Example

I → love → NLP

Each word is processed while remembering previous words.

---

8. Vanishing Gradient

The **Vanishing Gradient Problem** occurs when gradients become extremely small during training.

Effect

- Model forgets long-term information.
- Learning becomes slow.

---

9. Exploding Gradient

The **Exploding Gradient Problem** occurs when gradients become very large.

Effect

- Unstable training.
- Large weight updates.

---

10. LSTM (Long Short-Term Memory)

LSTM is an improved RNN that can remember information for long periods.

It solves the vanishing gradient problem better than a standard RNN.

---

11. Memory Cell

The **Memory Cell** stores important information over time.

It decides what information should be remembered.

---

12. Forget Gate

The **Forget Gate** removes information that is no longer useful.

Purpose

Keep only important information.

---

13. Input Gate

The **Input Gate** decides what new information should be stored in memory.

---

14. Output Gate

The **Output Gate** decides what information should be passed to the next time step.

---

15. GRU (Gated Recurrent Unit)

A **GRU** is a simpler version of LSTM.

Features

- Fewer gates
- Faster training
- Similar performance in many tasks

---

16. Bidirectional RNN

A **Bidirectional RNN** reads a sequence in **both directions**.

- Left → Right
- Right → Left

This provides better context.

---

17. Bidirectional LSTM

A **Bidirectional LSTM (BiLSTM)** combines:

- LSTM
- Bidirectional processing

It captures information from both past and future words.

Used in:

- Named Entity Recognition (NER)
- POS Tagging
- Question Answering

Module 8: Sequence-to-Sequence (Seq2Seq) Learning

What is Sequence-to-Sequence Learning?

Sequence-to-Sequence (Seq2Seq) is a deep learning model that converts one sequence into another sequence.

Input Sequence → Output Sequence

Applications

- Machine Translation
- Text Summarization
- Chatbots
- Question Answering

Example

English: I love AI
↓
French: J'aime l'IA

---

1. Seq2Seq

A **Seq2Seq** model consists of two main parts:

- Encoder
- Decoder

Flow:

Input Sentence
↓
Encoder
↓
Context Vector
↓
Decoder
↓
Output Sentence

---

2. Encoder

The **Encoder** reads the input sentence word by word and converts it into a numerical representation.

Example

Input:
I love NLP

The encoder learns the meaning of the sentence.

---

3. Decoder

The **Decoder** generates the output sentence one word at a time using information from the encoder.

Example

Input:
I love NLP

Output:

J'aime le NLP

---

4. Context Vector

A **Context Vector** is a fixed-size vector created by the encoder.

It contains the important information from the input sentence.

It acts as a summary of the input.

---

5. Teacher Forcing

Teacher Forcing is a training technique where the decoder receives the **correct previous word** instead of its own prediction.

Example

Correct sentence:

I love NLP

While predicting **love**, the decoder is given the correct previous word **I**.

Benefit

- Faster training
- Better learning

---

6. Beam Search

Beam Search keeps multiple best predictions at each step instead of only one.

Example

Beam Width = 3

Possible outputs:

I love AI
I love NLP
I enjoy NLP

The sequence with the highest probability is selected.

Benefit

- More accurate output
- Better than Greedy Decoding

---

7. Greedy Decoding

Greedy Decoding always chooses the most probable word at each step.

Example

I
↓
love
↓
NLP

Advantage

- Very fast

Disadvantage

- May miss the best overall sentence.

---

8. Exposure Bias

Exposure Bias occurs because:

- During training → Model sees the correct previous word (Teacher Forcing).
- During testing → Model uses its own previous prediction.

A wrong prediction can lead to more errors later.

Module 9: Attention Mechanism

What is Attention?

Attention is a mechanism that allows the model to focus on the **most important words** while generating each output word.

Instead of remembering everything equally, the model gives more importance to relevant words.

---

1. Why Attention?

In Seq2Seq models, the encoder compresses the entire sentence into **one context vector**.

For long sentences, this summary may lose important information.

Attention solves this problem by allowing the decoder to look back at the encoder outputs whenever needed.

Benefit

- Better understanding of long sentences.
- Improved translation and text generation.

---

2. Attention Score

An **Attention Score** measures how important each input word is for predicting the current output word.

Higher score = More attention.

---

3. Alignment

Alignment determines which input words are most related to the current output word.

Example

English : I love NLP
French : J'aime le NLP

While generating **love**, the model mainly aligns with **love** in the input.

---

4. Context Vector

In Attention, the **Context Vector** is created by combining encoder outputs using attention scores.

Unlike basic Seq2Seq, the context vector **changes for every output word**.

Dynamic context gives better performance than one fixed context vector.

---

5. Additive Attention

Also called **Bahdanau Attention**.

It uses a small neural network to calculate attention scores.

Features

- More flexible
- Works well for longer sequences
- Slightly slower

---

6. Multiplicative Attention

Also called **Luong Attention** or **Dot-Product Attention**.

It computes attention scores using the dot product of vectors.

Features

- Faster
- Computationally efficient

---

7. Global Attention

Global Attention considers **all input words** while generating each output word.

Example

Input:

I love learning NLP

The decoder can attend to every word.

---

8. Local Attention

Local Attention focuses only on a **small part** of the input sequence.

Instead of all words, it attends to nearby relevant words.

Benefit

- Faster
- Lower memory usage

Module 10: Part-of-Speech (POS) Tagging

What is POS Tagging?

Part-of-Speech (POS) Tagging is the process of assigning a grammatical label (tag) to each word in a sentence.

It tells whether a word is a **noun, verb, adjective, pronoun**, etc.

Example

Sentence:

The cat is sleeping.

Word	POS Tag
The	Determiner
cat	Noun
is	Verb
sleeping	Verb

---

1. POS Tagging

POS Tagging identifies the grammatical role of every word.

Applications

- Machine Translation
- Chatbots
- Grammar Checking
- Text Summarization
- Named Entity Recognition

---

2. POS Tags

A **POS Tag** is a label assigned to a word.

Common POS Tags

Tag	Meaning
Noun	Person, place, thing
Verb	Action word
Adjective	Describes a noun
Adverb	Describes a verb
Pronoun	Replaces a noun
Determiner	the, a, an
Preposition	in, on, at
Conjunction	and, but, or
Interjection	Wow!, Oh!

---

3. Universal POS Tags

The **Universal POS Tagset** is a simple and language-independent set of POS tags.

Some common tags:

Tag	Meaning
NOUN	Noun
VERB	Verb
ADJ	Adjective
ADV	Adverb
PRON	Pronoun
DET	Determiner
ADP	Preposition
AUX	Auxiliary Verb
NUM	Number
PUNCT	Punctuation

---

4. Penn Treebank Tags

The **Penn Treebank Tagset** is a detailed POS tagging standard for English.

Examples:

Tag	Meaning
NN	Noun
NNS	Plural Noun
NNP	Proper Noun
VB	Base Verb
VBD	Past Tense Verb
VBG	Verb ending in "ing"
JJ	Adjective
RB	Adverb
DT	Determiner
PRP	Personal Pronoun

---

5. Rule-Based POS Tagging

Uses manually written grammar rules.

Example

If a word ends with "ly"
→ Adverb

Advantages

- Easy to understand
- No training data needed

Disadvantages

- Difficult to maintain
- Less accurate

---

6. Statistical POS Tagging

Uses probabilities learned from labeled data.

It predicts the most likely POS tag based on context.

Example

Book a ticket.

Here, **Book** is predicted as a **Verb**.

---

7. Hidden Markov Model (HMM)

HMM is a statistical model used for POS tagging.

It predicts the most probable sequence of tags using probabilities.

Example

The dog runs.

Possible tags:

DT → NN → VB

---

8. Conditional Random Field (CRF)

CRF is a statistical sequence model that predicts POS tags by considering neighboring words and tags together.

Advantages

- Better context understanding
- Higher accuracy than HMM

---

9. Neural POS Tagging

Uses **Deep Learning** models such as:

- RNN
- LSTM
- BiLSTM
- Transformer

Advantages

- Learns automatically
- High accuracy
- Handles context better

Module 11: Named Entity Recognition (NER)

What is Named Entity Recognition (NER)?

Named Entity Recognition (NER) is the process of identifying and classifying important entities in text.

It answers: **"Who?", "Where?", "When?", "How much?"**

Example

Elon Musk founded Tesla in California in 2003.

Word	Entity
Elon Musk	Person
Tesla	Organization
California	Location

Word	Entity
2003	Date

---

1. Named Entities

Named Entities are real-world names or objects mentioned in text.

Common entity types:

- Person
- Organization
- Location
- Date
- Time
- Money
- Quantity

---

2. Person

A **Person** entity represents a person's name.

Examples

Virat Kohli
Sachin Tendulkar
Albert Einstein

---

3. Organization

An **Organization** is the name of a company, institution, or group.

Examples

Google
Microsoft
OpenAI

---

4. Location

A **Location** is a place.

Examples

India
Mumbai
New York

---

5. Date

A **Date** represents a calendar date.

Examples

15 August 1947
1 January 2026
Monday

---

6. Time

A **Time** entity represents a specific time.

Examples

10:30 AM
6 PM
Midnight

---

7. Money

A **Money** entity represents monetary values.

Examples

₹500
$100
€50

---

8. Quantity

A **Quantity** represents measurable values.

Examples

5 kg
10 km
2 liters

---

9. Rule-Based NER

Uses predefined rules and dictionaries to identify entities.

Example

Words starting with capital letters
→ Possible Person Name

Advantages

- Simple
- No training required

Disadvantages

- Cannot handle unseen entities well.

---

10. Statistical NER

Uses machine learning models trained on labeled data.

Examples:

- HMM
- CRF

Advantages

- Better accuracy
- Learns patterns from data

---

11. Deep Learning NER

Uses neural networks to identify entities.

Common models:

- BiLSTM
- BiLSTM + CRF
- BERT
- Transformer

Advantages

- High accuracy
- Understands context
- Handles unseen entities better

---

12. Entity Linking (Concept)

Entity Linking connects a recognized entity to its correct real-world identity in a knowledge base.

Example

Apple

Possible meanings:

- Apple (Company)
- Apple (Fruit)

Entity Linking uses the **context** to choose the correct one.

Module 12: Parsing

What is Parsing?

Parsing is the process of analyzing the grammatical structure of a sentence.

It helps NLP systems understand **how words are related**.

Example

The cat eats fish.

Parsing identifies:

- Subject → The cat

- Verb → eats
- Object → fish

---

1. Syntactic Parsing

Syntactic Parsing analyzes the grammar of a sentence and builds its structure.

Purpose

- Check grammar
- Understand relationships between words

---

2. Constituency Parsing

Constituency Parsing divides a sentence into phrases (constituents).

Example

The smart student studies NLP.
Sentence
├── Noun Phrase (The smart student)
└── Verb Phrase (studies NLP)

Focus: Phrase structure.

---

3. Dependency Parsing

Dependency Parsing shows how words depend on each other.

Example

She reads books.
reads
├── She
└── books

Focus: Word-to-word relationships.

---

4. Parsing Algorithms

Parsing Algorithms are methods used to build parse trees.

Popular algorithms:

- CKY Parser
- Earley Parser

---

5. CKY Parsing

CKY (Cocke–Kasami–Younger) Parser is a dynamic programming algorithm used for constituency parsing.

Features

- Fast and efficient
- Works with Context-Free Grammars (CFG)
- Produces parse trees

---

6. Earley Parser (Concept)

The **Earley Parser** is a parsing algorithm that works with almost any Context-Free Grammar.

Features

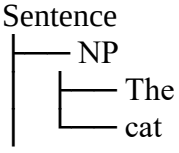
- Handles ambiguous grammar
- Suitable for natural language
- More flexible than CKY

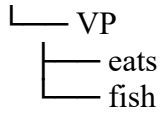
---

7. Parse Trees

A **Parse Tree** is a tree representation of a sentence's grammatical structure.

Example





It shows how words combine to form phrases and sentences.

Module 13: Text Classification

What is Text Classification?

Text Classification is the process of assigning one or more labels to a text.

Input Text → Predicted Category

Examples

- Spam Detection
- Sentiment Analysis
- Topic Classification
- Intent Detection

---

1. Binary Classification

Binary Classification classifies text into **two classes**.

Examples

- Spam / Not Spam
- Positive / Negative
- Fake / Real

---

2. Multi-Class Classification

Multi-Class Classification assigns **one label** from **more than two classes**.

Example

News Article

Possible classes:

- Sports
- Politics
- Technology
- Business

Only **one** class is selected.

---

3. Multi-Label Classification

Multi-Label Classification assigns **multiple labels** to the same text.

Example

Article:

AI is transforming healthcare and education.

Labels:

- AI □
- Healthcare □
- Education □

A text can belong to multiple categories.

---

4. Naive Bayes

Naive Bayes is a probability-based classification algorithm.

Features

- Fast
- Simple
- Works well for text classification

Commonly used for:

- Spam Detection
- Sentiment Analysis

---

5. Logistic Regression

Logistic Regression predicts the probability of each class.

Features

- Simple
- Fast
- Good baseline model

Used for:

- Binary classification
- Multi-class classification

---

6. Support Vector Machine (SVM)

SVM finds the best boundary (decision boundary) between different classes.

Features

- High accuracy
- Works well with high-dimensional text data

---

7. Decision Trees

A **Decision Tree** classifies text by making a series of decision rules.

Example

Contains "Free"?
↓
Yes
↓
Spam

Easy to understand and visualize.

---

8. Random Forest

A **Random Forest** combines many Decision Trees.

Final prediction = Majority vote of all trees.

Benefits

- More accurate
- Reduces overfitting

---

9. Neural Networks

Neural Networks learn complex patterns from text automatically.

Examples:

- Feedforward Networks
- CNN
- RNN
- LSTM
- Transformers

They generally provide the best performance on large datasets.

---

10. Spam Detection

Classifies emails or messages as:

- Spam
- Not Spam

Example

Congratulations! You won ₹1,00,000.

Prediction:

Spam

---

11. Topic Classification

Assigns a topic to a document.

Example

Article about football

↓

Sports

---

12. Intent Classification

Identifies the user's intention.

Examples

Book a flight.

Intent:

Book Flight

Cancel my order.

Intent:

Cancel Order

---

13. Toxic Comment Detection

Detects harmful or offensive comments.

Examples:

- Hate speech
- Abusive language
- Offensive content
- Cyberbullying

Used by:

- Social media platforms
- Online forums
- Gaming platforms

Module 23: NLP Evaluation

What is NLP Evaluation?

NLP Evaluation is the process of measuring how well an NLP model performs.

It helps compare models and choose the best one.

Examples

- Is the classifier accurate?
- Is the translation correct?
- Is the summary meaningful?

---

1. Classification Metrics

Classification Metrics evaluate models that predict categories or labels.

Used in:

- Spam Detection
- Sentiment Analysis
- Text Classification
- Named Entity Recognition

Common metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

---

2. Accuracy

Accuracy is the percentage of correct predictions.

Formula

Accuracy = Correct Predictions / Total Predictions

Example

Total emails = **100**

Correctly classified = **95**

Accuracy:

95%

Best when

- Classes are balanced.

---

3. Precision

Precision measures how many predicted positive results are actually positive.

Formula

Precision = True Positives / (True Positives + False Positives)

Example

Predicted Spam = **20**

Actual Spam = **18**

Precision:

18/20 = 90%

High Precision

- Fewer false positives.

---

4. Recall

Recall measures how many actual positive cases are correctly identified.

Formula

Recall = True Positives / (True Positives + False Negatives)

Example

Actual Spam = **25**

Detected Spam = **20**

Recall:

20/25 = 80%

High Recall

- Fewer missed positive cases.

---

5. F1-Score

F1-Score is the harmonic mean of Precision and Recall.

It balances both metrics.

Properties

- High only when both Precision and Recall are high.
- Useful for imbalanced datasets.

---

6. Confusion Matrix

A **Confusion Matrix** shows the number of correct and incorrect predictions.

Actual / Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Meaning

- **TP** → Correct positive prediction
- **TN** → Correct negative prediction
- **FP** → Incorrect positive prediction
- **FN** → Missed positive prediction

---

7. Perplexity

Perplexity evaluates **Language Models**.

It measures how well the model predicts the next word.

Interpretation

- Lower Perplexity → Better model
- Higher Perplexity → Poor model

Example

Model	Perplexity
Model A	12
Model B	25

Model A performs better.

---

8. BLEU (Bilingual Evaluation Understudy)

BLEU evaluates **Machine Translation** by comparing the generated translation with one or more reference translations.

Example

Reference:

I love NLP.

Generated:

I like NLP.

BLEU measures how closely they match.

Higher BLEU

- Better translation quality.

---

9. ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE evaluates **Text Summarization**.

It compares the generated summary with the reference summary.

Example

Original:

Artificial Intelligence is transforming healthcare.

Reference Summary:

AI is transforming healthcare.

Generated Summary:

AI improves healthcare.

ROUGE measures how much important information is shared.

Higher ROUGE

- Better summary quality.

---

10. METEOR (Metric for Evaluation of Translation with Explicit ORdering)

METEOR evaluates translation by comparing:

- Exact word matches
- Stem matches
- Synonyms
- Word order

It often correlates better with human judgment than BLEU.

Higher METEOR

- Better translation quality.

---

11. CIDEr (Concept)

CIDEr (Consensus-based Image Description Evaluation) evaluates **image captions**.

It compares the generated caption with multiple human-written captions.

Example

Image Caption:

A dog is running in a park.

The closer it is to human captions, the higher the CIDEr score.

Used in

- Image Captioning
- Vision-Language Models